

Relatório — ADO 2: Simulador de Sistemas Mecânicos

Disciplina: Física — Práticas Extensivas

Professor: Dr. Ettore Baldini-Neto

Integrantes: Álvaro Cereja de Souza, Flávio Miranda, Gabriel de Lima Dourado.

1. Introdução

Este trabalho implementa uma interface gráfica interativa em **p5.js** (JavaScript) para simular dois sistemas mecânicos clássicos regidos pela Segunda Lei de Newton ($\Sigma F = ma$):

1. **Plano inclinado com atrito e tração** (sistema 3.1 do roteiro);
2. **Força centrípeta em movimento circular uniforme** (sistema 3.2), incluindo a extensão opcional do carro em curva plana.

Nosso grupo optou por implementar os sistemas **3.1 e 3.2**, ficando o sistema 3.3 (força de arrasto) de fora do escopo.

Em todos os casos, as equações de movimento possuem **solução analítica fechada** — nenhum método de integração numérica (Euler, Runge-Kutta etc.) foi empregado. As grandezas são calculadas diretamente pelas fórmulas fechadas fornecidas na Seção 3 do roteiro.

2. Equações utilizadas e derivação

2.1 Sistema 3.1 — Plano inclinado com atrito e tração

Diagrama de corpo livre. Considere um bloco de massa m_1 sobre um plano inclinado de ângulo θ , ligado por um fio ideal (inextensível e sem massa) que passa por uma polia sem atrito e sustenta uma massa suspensa m_2 . Sejam μ_s o coeficiente de atrito estático e μ_k o cinético entre o bloco e o plano.

Decompondo as forças sobre m_1 :

- **Direção perpendicular ao plano (equilíbrio):**
 $N - m_1 \cdot g \cdot \cos \theta = 0 \Rightarrow N = m_1 \cdot g \cdot \cos \theta$
- **Direção paralela ao plano (positivo = subida):**
 $T - m_1 \cdot g \cdot \sin \theta - f_{\text{atrito}} = m_1 \cdot a$

Sobre m_2 (vertical, positivo = descida):

$$m_2 \cdot g - T = m_2 \cdot a$$

Somando as duas equações e eliminando a tração T , define-se a **força motriz**:

$$F_{\text{motriz}} = m_2 \cdot g - m_1 \cdot g \cdot \sin \theta$$

Regime de equilíbrio estático. O sistema permanece em repouso enquanto a força motriz não superar o atrito estático máximo:

$$|F_{\text{motriz}}| \leq \mu_s \cdot N \Rightarrow a = 0$$

Regime de movimento. Caso contrário, o atrito cinético se opõe ao movimento e a aceleração é:

$$a = [F_{\text{motriz}} - \mu_k \cdot N \cdot \text{sign}(F_{\text{motriz}})] / (m_1 + m_2)$$

A tração no fio é obtida isolando m_2 : $T = m_2 \cdot (g - a)$.

Cinemática (MRUV). Sendo a constante após definido o regime:

$$\begin{aligned}x(t) &= x_0 + v_0 \cdot t + \frac{1}{2} \cdot a \cdot t^2 \\v(t) &= v_0 + a \cdot t\end{aligned}$$

2.2 Sistema 3.2 — Força centrípeta

Movimento Circular Uniforme (MCU). Uma massa m presa a um fio de comprimento R gira com velocidade tangencial constante v . As equações fechadas são:

$$\omega = v / R \text{ (velocidade angular)}$$

$$x(t) = R \cdot \cos(\omega \cdot t), y(t) = R \cdot \sin(\omega \cdot t)$$

$$F_{cp} = m \cdot v^2 / R = m \cdot \omega^2 \cdot R \text{ (força centrípeta)}$$

$$T = 2\pi / \omega \text{ (período)}$$

Extensão opcional — carro em curva plana. A força centrípeta é fornecida pelo atrito estático entre pneu e pista. A velocidade máxima antes de derrapar é:

$$v_{MAX} = \sqrt{\mu g R}$$

3. Interface e controles implementados

A aplicação foi construída em **p5.js** (editor online), com os seguintes componentes:

- **Menu principal (RadioButtons)** para escolher entre "Plano inclinado" e "Força centrípeta".
- **Submenu** (visível apenas no sistema 3.2) para alternar entre "MCU simples" e "Carro em curva".
- **Sliders interativos**, criados dinamicamente conforme o sistema ativo:
 - **Sistema 3.1:** m_1 , m_2 , θ , μ_s , μ_k
 - **Sistema 3.2 (MCU):** R , v , m
 - **Sistema 3.2 (Carro):** R , m , μ
- **Painel numérico atualizado em tempo real** exibindo: para o 3.1 — aceleração, tração, força motriz, normal e o regime (MOVIMENTO /

EQUILÍBRIO ESTÁTICO); para o MCU — ω , F_c e T ; para o carro — v_{max} e $F_{c,\text{max}}$.

- **Gráfico principal:**

- 3.1: curvas $x(t)$ (posição, azul) e $v(t)$ (velocidade, laranja) em função do tempo.
- 3.2: trajetória circular com **ponto animado** e **seta vetorial verde** indicando a força centrípeta apontando para o centro.

Todos os sliders atualizam o gráfico **em tempo real**, sem necessidade de reexecutar o código.

4. Sistema 3.1 — Determinação do μ crítico

Para determinar experimentalmente o valor crítico de atrito estático (transição MOVIMENTO → EQUILÍBRIO), configuramos na interface:

Parâmetro	Valor
m1	5 kg
m2	4 kg
Teta	30°
Mi cinético	0,25 (fixo)

Variando μ_s pelo slider, observamos a transição de regime em:

- **μ_s crítico observado na interface: 0,35**
- **Valor teórico esperado:** $\mu_{s_crit} = |F_{\text{motriz}}| / N$
 - $F_{\text{motriz}} = 4,0 \cdot 9,81 - 5,0 \cdot 9,81 \cdot \sin(30^\circ) = 14,715 \text{ N}$
 - $N = 5,0 \cdot 9,81 \cdot \cos(30^\circ) = 42,478 \text{ N}$
 - $\mu_{s_crit} = 14,715 / 42,478 \approx 0,346$

Discussão. O valor experimental (0,35) coincide com o valor teórico (0,346) dentro da precisão do passo do slider (0,01). Isso confirma que a implementação reproduz corretamente o critério de equilíbrio $|F_{\text{motriz}}| = \mu_s \cdot N$ previsto pela 2ª Lei de Newton.

5. Sobre o item 5.3 do roteiro (sistema 3.3)

O item 5.3 do roteiro solicita uma comparação entre arrasto linear e quadrático — porém esse conteúdo refere-se ao sistema 3.3, que nosso grupo não implementou, em conformidade com a Seção 7 ("cada grupo deve escolher no mínimo 2 dos 3 sistemas"). Como optamos pelos sistemas 3.1 e 3.2, este item não se aplica ao presente trabalho.

6. Aplicações reais dos sistemas simulados

Sistema 3.1 — Plano inclinado com atrito e tração. Modela sistemas de elevação com contrapeso (elevadores, guindastes), transporte de cargas em rampas industriais, tirolesas e planos inclinados em geral. A distinção entre regime de equilíbrio e de movimento é fundamental no dimensionamento de sistemas de freio, contrapeso e motores.

Sistema 3.2 — Força centrípeta. Aplica-se a satélites em órbita circular, rotores de centrífugas, movimento de partículas em aceleradores de baixa energia e, no caso do carro em curva plana, ao projeto de rodovias: o raio mínimo, a inclinação de pistas (superelevação) e os limites de velocidade segura em diferentes condições de atrito são calculados exatamente pela relação: $v_{MAX} = \sqrt{\mu g R}$.

7. Organização do código

O código em p5.js está organizado em **três camadas**:

1. **Física (funções puras):** `calcularAceleracaoPlano`, `calcularTracaoPlano`, `calcularMCU`, `calcularCarroCurva`. Não dependem de nenhum recurso gráfico e podem ser testadas isoladamente.
2. **Interface (desenho e eventos):** funções `desenhar*` (desenho de cada componente) e callbacks de evento (`mousePressed`, `mouseDragged`, `mouseReleased`, `keyPressed`).
3. **Loop de renderização:** a função nativa `draw()` do p5.js, executada 60 vezes por segundo, redesenha todos os elementos e avança a fase de animação do sistema 3.2 de forma proporcional à velocidade angular real: $\omega = \frac{v}{R}$.

Cada fórmula tem comentário indicando a lei física de origem, conforme exigido no roteiro.